

TLDR

To build and deploy the OpenSearch load-testing environment with a graphical user interface, run:

```
bash start-ui.sh rebuild
```

This command first builds the UI and opens an interface where you can configure and launch load tests with a few clicks. When you click the "Start Load Test" button, the script builds all required Docker images and starts the complete OpenSearch load-tester stack.

To remove everything created during the build process (containers, volumes, and images), click the "clean" button.

Requirements

The following tools must be installed:

- Java 25
- docker
- docker-compose
- make

Docker Stack

The load-testing environment consists of the following containers specified in `docker-compose.yaml`:

Container name	Build context / image (assumption)
testdata-generator	./testdata-generator (built from source)
load-generator	./load-tester (built from source)
metrics-reporter	./metrics-reporter (built from source)
test-target-opensearch	Pulled from Docker Hub
alloy	Pulled from Docker Hub
loki	Pulled from Docker Hub
grafana	Pulled from Docker Hub

Alloy is used as a log shipper to send OpenSearch logs to Loki, which Grafana then visualizes. Grafana can be accessed on port `3000` of the host machine. We provide pre-configured dashboards for monitoring OpenSearch performance during load tests.

The configuration files for Grafana and Loki and Alloy are located in the `deploy` directory.

Dockerfiles

Each Spring Boot container has its own Dockerfile.

```
./testdata-generator/Dockerfile
./load-tester/Dockerfile
./metrics-reporter/Dockerfile
```

All applications use a version pinned multi stage Docker build. The build stage uses `maven:3.9-eclipse-temurin-25` as base image and the runtime stage uses a minimal image based on `eclipse-temurin:25-jre-alpine`

The application is placed in the `/app` directory and executed using the non-root user `user` to reduce privilege escalation risks.

Container orchestration

All images are managed through a single `docker-compose.yaml` file and are configured via a `.env` file. The containers run inside the Docker bridge network `opensearch-loadtester-network` and communication takes place over HTTP using Docker DNS resolution.

Deployment

You can choose between a graphical and an interactive deployment process via make on your CLI.

Graphical

The source code of the UI is located in the `ui` directory and can be built with the provided bash script `start-ui.sh`: Or you can directly run:

```
mvn clean install -DskipTests
mvn -f ui/pom.xml clean javafx:link
cp -r ./ui/target/LoadTesterUI .
./LoadTesterUI/bin/LoadTesterUI
```

Make

You can also use `make build` to build the container images. In the background it runs:

```
docker-compose build testdata-generator
docker-compose build load-generator
docker-compose build metrics-reporter
```

To clean up all created containers, volumes, and built images, run you can use `make stop` and `make clean` which runs:

```
docker-compose down
```

and

```
docker-compose down --volumes --rmi local --remove-orphans
```